

Free Value Or Else

Steve Downey <sdowney@gmail.com>
Corentin Jabot <corentin.jabot@gmail.com>

Document #: D4270R1
Date: 2026-09-10
Project: Programming Language C++
Audience: LEWG

Abstract

`value_or` has two defects, and this paper carries the fix for each. The member half, from [\[P3413R0\]](#), adds `value_or_construct` and `value_or_else` to `std::optional` and `std::expected`, so that the alternative is built only when it is needed. The free half adds free function overloads of `value_or` — and the related `reference_or`, `or_invoke`, and `or_construct` — over any nullable type, such as `std::optional`, `std::expected`, and pointers to objects, computing the type of the result correctly, via `common_type` and `common_reference`, as the member `std::optional::value_or` could not. The two were separate papers until LEWG asked for one. They fix the cost of the alternative and the type of the answer, they compose, and they are severable.

Contents

1	Comparison table	2
2	Motivation	2
3	Design	3
4	P3413R0, and the two halves	8
5	Principles for Reification of Design	9
6	Implementation Experience	10
7	Proposal	11
8	Wording	12
9	Impact on the standard	15
10	Acknowledgments	15
11	Document history	16
	References	16

Changes Since Last Version

- **R1** Unified with [\[P3413R0\]](#) at LEWG's request: Corentin Jabot joins as co-author, and the member functions `value_or_construct` and `value_or_else` are folded into the proposal, the wording, and the implementation. The members are specified over `optional<T&>`, which P3413R0 deferred and which is now C++26, rather than deferred. Independently of the unification, an rvalue nullable now moves its payload, which splits the models by ownership, and the asks to LEWG are cut to two.

1 Comparison table

1.1 Raw pointer with a manual null check

The fundamental nullable type is the raw pointer, and the fundamental idiom for providing a default is the conditional expression guarded by a null check. The free function gives that pattern a name, removes the repetition of the checked entity, and computes the type of the result correctly.

```
Cat* cat = find_cat("Fido");  
return cat ? *cat : Cat{"Default"};  
  
Cat* cat = find_cat("Fido");  
return std::value_or(cat, Cat{"Default"});
```

1.2 The fallback that is built anyway

The argument to `value_or` is an ordinary function argument, constructed on every call and destroyed unused on every engaged one. The member `value_or_construct` takes the fallback's constructor arguments instead, and builds nothing when the optional is engaged; `value_or_else` defers an arbitrary computation the same way.

```
std::optional<Config> o = load();  
// Config built even when o is engaged  
return o.value_or(Config{"etc/default"});  
  
std::optional<Config> o = load();  
// built only when o is disengaged  
return o.value_or_construct("etc/default");
```

1.3 The `value_or` truncation pitfall

Because `std::optional<T>::value_or` converts its argument to `T`, a perfectly reasonable-looking sentinel silently truncates. The free function computes the common type of the contained value and the alternative, and converts both *outward* to that type instead of forcing the alternative *inward* through `T`.

```
std::optional<std::uint16_t> o = get();  
auto v = o.value_or(-1);  
// v is uint16_t{65535}  
  
std::optional<std::uint16_t> o = get();  
auto v = std::value_or(o, -1);  
// v is int; -1 is preserved
```

1.4 Reference alternative without copies

When both the contained value and the alternative are references to long-lived objects, the result should be a reference. Member `value_or` cannot do this; `reference_or` can, and statically rejects the cases where the result would dangle.

```
std::optional<Logger&> opt = get_logger();  
Logger& l =  
    opt ? *opt : default_logger();  
  
std::optional<Logger&> opt = get_logger();  
Logger& l = std::reference_or(  
    opt, default_logger());
```

2 Motivation

`std::optional<T>::value_or` answers the most common question asked of an optional — “the value, or this instead” — and gets two things wrong. It answers with the wrong type, funneling both the contained value and the alternative through `T`. And it pays for the alternative whether or not it is wanted, because the alternative is an ordinary function argument. The two defects are independent, they have been pursued independently, and this paper carries the fix for each.

2.1 The type of the answer

`value_or` returns `T`, and it requires the alternative to be convertible to `T`. Every use therefore funnels both the contained value and the alternative through `T`, whether or not `T` is the right type for the result. The consequences are familiar: sentinels truncate (`optional<uint16_t>.value_or(-1)` is 65535), wider alternatives narrow, and reference results are simply impossible, so `value_or` forces a copy even when both candidate results are references to long-lived objects. It answers the wrong question correctly.

This is not fixable in place. Changing the return type of a member function that has been shipping since C++17 is both an API and an ABI break, and the `&&`-qualified overload set compounds the problem. During the work on `std::optional<T&>` [P2988R12] the question was reopened in earnest, because for an optional reference the cost of the wrong answer is no longer a pessimization but a wrong program: returning `T` from `optional<T&>::value_or` copies, returning `T&` dangles when the alternative is a temporary. After extensive discussion in LEWG, P2988 concluded that “there is no particularly wonderful solution for `value_or` that does not involve a time machine,” adopted the least objectionable member behavior — return `remove_cv_t<T>`

by value — and explicitly forecast this paper: free functions over all types modeling an optional-like concept, where the return type can finally be computed correctly.

Existing practice diverges because there is no one member-function answer that satisfies everyone:

Implementation	Behavior
Standard	<code>optional<T>::value_or</code> returns a <code>T</code>
Boost	<code>optional<T>::value_or</code> returns a <code>T</code> <code>optional<T&>::value_or</code> returns a <code>T&</code>
TartanLlama	<code>optional<T>::value_or</code> returns a <code>T</code> <code>optional<T&>::value_or</code> returns a <code>T</code>
Flux	returns the type of the ternary, similar to <code>common_reference</code>
Think-Cell	returns <code>common_reference</code> , with some caveats

The tools to compute the right answer postdate `optional`. `std::common_type` existed but was not used; `std::common_reference` arrived with ranges in C++20; and `std::reference_constructs_from_temporary_v` [P2255R2] arrived in C++23, making it possible for the first time to *statically reject* the dangling cases that made reference-returning `value_or` untenable. The conditional expression `b ? *m : u` has computed the common type all along; the library can now name that computation, add the lifetime check the core language does not perform, and do so once, for every nullable type.

That last point is the generalization this paper takes from [P1255R13]: being “contextually convertible to `bool` and dereferenceable” is a protocol, not a property of `std::optional`. Raw pointers, `std::shared_ptr`, `std::unique_ptr`, `std::expected<T,E>`, and `std::optional<T&>` all speak it. Each of them today requires either a hand-written conditional or a type-specific spelling of “the value, or this instead.” A single constrained free function family covers them all, with one set of semantics to learn and one place to get the corner cases right.

2.2 The cost of the alternative

The second defect is independent of the first, and older in the committee’s record. `o.value_or(Config"/etc/default")` constructs a `Config` on every call and destroys it unused on every engaged one. The parameter is an ordinary function parameter, so the alternative is evaluated whether or not it is reached, and the cost is unbounded: an expensive construction, an allocation, a lookup, a computation whose result is thrown away. The workaround is the conditional expression the member was supposed to replace, and callers who care about the cost write it.

The cure has been proposed before. [P2218R0], by Marc Mutz, proposed `value_or_construct`, taking the fallback’s constructor arguments rather than a constructed fallback, so that nothing is built on the engaged path; it did not proceed at the time. [P3413R0] revived it, added `value_or_else` for the general case of a lazily evaluated nullary callable, and specified both on `std::optional` and `std::expected`. Those members are the member half of this paper, and their wording is reproduced below.

Laziness is orthogonal to the type of the result. `value_or_construct` and `value_or_else` still return `T`, and still require the alternative to converge on `T`. They fix what they set out to fix, on the two types that have the member to grow.

2.3 Two defects, two shapes

The fixes take different shapes because the defects are of different kinds. The cost is a defect in the parameters, and a new member name fixes it in place: `value_or_construct` and `value_or_else` sit beside `value_or` on `optional` and `expected`, spelled the way the member they join is spelled, and nothing existing changes meaning. The type is a defect in the signature, and no member can fix it — the return type of a member shipping since C++17 cannot change, a corrected second member would have to be written and kept in sync by hand on every class that has members at all, and a raw pointer has none (see Why free functions, below).

So this paper proposes each defect’s fix in the shape that defect admits: a pair of members, and a family of free functions over the protocol. Neither forecloses the other, and neither depends on the other.

3 Design

The proposal has two halves. The free half is four function templates over a `nullable` concept:

```

template <class T>
concept nullable = requires(const std::remove_reference_t<T>& t) {
    bool(t);
    *(t);
};

// The type *m yields, carrying the value category of the nullable through
// to its payload. Exposition only in the wording.
template <class T>
using deref_t = decltype(*std::declval<T>());

template <nullable T, class U,
        class R = std::common_reference_t<deref_t<T>, U&&>>
constexpr auto reference_or(T&& m, U&& u) -> R;

template <nullable T, class U,
        class R = std::common_type_t<deref_t<T>, U&&>>
constexpr auto value_or(T&& m, U&& u) -> R;

template <nullable T, class I,
        class R = std::common_type_t<deref_t<T>,
                                std::invoke_result_t<I>>>
constexpr auto or_invoke(T&& m, I&& invocable) -> R;

template <class Ret = void, nullable T, class... Args,
        class R = std::conditional_t<std::is_void_v<Ret>,
                                std::remove_cvref_t<deref_t<T>>, Ret>>
constexpr R or_construct(T&& m, Args&&... args);

template <class Ret = void, nullable T, class E, class... Args,
        class R = std::conditional_t<std::is_void_v<Ret>,
                                std::remove_cvref_t<deref_t<T>>, Ret>>
constexpr R or_construct(T&& m, std::initializer_list<E> il, Args&&... args);

```

All four have the same engaged-path behavior, producing `*m` converted to `R`, and differ in how the alternative is supplied (a value, a value bound as a reference, a nullary invocable, constructor arguments) and in how `R` is computed (`common_type`, `common_reference`, or the element type itself).

The member half is `value_or_construct` and `value_or_else` on `std::optional` and `std::expected`, as [P3413R0] specified them:

```

template <class... Args>
constexpr T value_or_construct(Args&&... args) const&

template <class U, class... Args>
constexpr T value_or_construct(std::initializer_list<U> il,
                                Args&&... args) const&

template <class F>
constexpr T value_or_else(F&& f) const&

// and the corresponding <del>E</del> overloads, on optional and expected alike

```

Both return `T`, as the member `value_or` beside them does, and neither constructs anything on the engaged path. The sections that follow give the design of the free half, where the open choices are; the member half is taken up under The member half, below.

3.1 The nullable concept

The concept is the “maybe protocol” of [P1255R13]: an object is nullable if it is contextually convertible to `bool` and dereferenceable. The requirements are stated on a `const` lvalue: observation must not require mutable access. The reference is stripped before the `const` is applied, so the rule holds for the deduced `T` of an lvalue argument as much as for an rvalue; spelled `const T` it would not, since `const` applied to a reference type is discarded. A type whose `operator bool` or `operator*` is non-`const` does not model `nullable`. Asking “the value or a default” is a query. A type that can not be queried through `const` access is not answering it.

Types verified to model the concept: `std::optional<T>`, `std::expected<T,E>`, raw object pointers `T*`, `std::shared_ptr<T>`, `std::unique_ptr<T>`, and `std::optional<T&>` as specified by [P2988R12]. Types verified not to model it: arithmetic types and `bool` (contextually convertible to `bool` but not dereferenceable), `std::string` and the containers (neither), `std::nullptr_t`, `void`, and types providing only one of the two operations. Iterators in general do not model `nullable` because they are not contextually convertible to `bool`. Correctly so: an iterator’s validity is not a property the iterator itself can report.

One refinement remains open. P1255’s `nullable_object` also requires that the result of dereferencing be an equality-preserving *object*, which excludes function pointers; the minimal concept above admits them (a function pointer is contextually convertible to `bool` and dereferenceable), and they are then rejected only downstream, when `common_type` fails. We keep the concept minimal. Requiring `is_object_v` of the referent would move that rejection to the constraint, which reads better in a diagnostic, at the cost of a concept users can no longer reproduce from its one-line statement.

3.2 Exposition-only, or a named public concept?

We propose `std::nullable` as a named, public concept, usable outside the standard library.

The protocol is vocabulary. [P2988R12] forecast these functions “over all types modeling optional-like, concept `std::maybe`”, and [P1255R13] is titled, in part, “a concept to constrain maybes” — the concept has been treated as a deliverable in its own right throughout this line of work. Users writing their own functions over nullable types (a `value_or` variant with different policy, an algorithm taking a range of nullables) need the same constraint, and without a public name each codebase re-derives it, and not always the same way. A named concept can be tested directly (`static_assert(std::nullable<T>)`), appears by name in diagnostics, and fixes the meaning of “optional-like” across the ecosystem once.

There is a principled reason not to. A named standard concept is forever, and a public name can never be demoted or meaningfully tightened, while an exposition-only one can be promoted later without breaking anything. The library has a directly analogous precedent: *boolean-testable*, among the most-used constraints in the standard, is exposition-only for that reason. However, the asymmetry cuts the other way here. A constraint nobody can name is one every codebase re-derives, and the re-derivations are what diverge; *boolean-testable* is plumbing for other concepts, where `nullable` is the thing users want to write down. With [P1255R13] concluded (see the discussion of `yield_if` below), this paper is the concept’s sole remaining home, so there is no cross-paper coordination to wait on.

The semantics a public name has to carry are stated with it: the contextual conversion is equality-preserving and does not modify the object; `*t` may be evaluated only when that conversion yields `true`; and both are valid on a `const lvalue`. LEWG may of course take the name back and make the concept exposition-only, which costs nothing but the spelling.

3.3 Return type computation

`deref_t<T>` is `decltype(*declval<T>())`, the type `*m` yields, with the value category of the nullable carried through. For a nullable passed as an lvalue with an `int` payload it is `int&`; for a `const optional<int>` it is `const int&`; for `const int*` it is `const int&`; and for an rvalue `optional<int>` it is `int&&`, on which see the next section. The return type is then the common type (for `value_or` and `or_invoke`) or the common reference (for `reference_or`) of that dereference type and the alternative.

For `value_or`, `common_type` decays, so the result is a prvalue, and mixed-type uses `promote` instead of truncating:

nullable T	alternative U	result R
<code>optional<int></code>	<code>int</code>	<code>int</code>
<code>expected<int,E></code> , <code>int*</code> , <code>shared_ptr<int></code> , <code>unique_ptr<int></code>	<code>int</code>	<code>int</code>
<code>optional<int></code>	<code>long</code>	<code>long</code>
<code>optional<int></code>	<code>double</code>	<code>double</code>
<code>optional<uint16_t></code>	<code>int</code> (e.g. -1)	<code>int</code>
<code>optional<string></code>	<code>string</code>	<code>string</code>

This is the type the equivalent conditional expression would have produced, which is the point: `value_or(m, u)` is the library spelling of `m ? *m : u`, not of `m ? *m : T(u)`.

3.4 Value categories, and what an rvalue nullable means

The value category of the nullable reaches its payload. `deref_t<T>` is `decltype(*declval<T>())`, so an expiring nullable dereferences to an expiring payload, and the engaged path moves rather than copies: `value_or(std::move(opt), u)` matches `std::move(opt).value_or(u)`. The free functions do not lose the member's efficiency.

That divides the models, and the division is the point, because it is not an accident of specification. `std::optional` and `std::expected` contain their value. An rvalue of one is an expiring object whose payload expires with it, and `operator*` is ref-qualified to say so. A pointer contains nothing. `T*`, `std::shared_ptr`, and `std::unique_ptr` are handles; an rvalue handle is an expiring handle, and says nothing about the lifetime of the referent, which ordinarily outlives it. `T&` is the truthful answer for all three, and they go on copying.

`std::optional<T&>` settles which property is doing the work. It is an `optional`, and it dereferences to `T&`, because it does not own its referent either. The split is ownership, not optional-ness. Each nullable reports what its own value category implies about its payload, and every one of them is right.

The alternative is to compute the dereference type from `T&` always, as `std::iter_reference_t` does, so that the functions observe and never consume. It buys a rule that fits in a sentence — the result is what the lvalue conditional expression would have produced — and three smaller things: the return type stops depending on the value category of the nullable; `reference_or` over an rvalue `optional<int>` stays `int&` instead of becoming `const int&`, which is the common reference of `int&&` and `int&`; and there is no moved-from path to reason about under the dangling analysis of `reference_or`. However, it buys them by making every owning nullable copy a payload that is going away, which is the wrong answer stated uniformly. We would rather be correct and pay the ergonomics.

The behavior is pinned either way. The engaged path is copy-counted for `optional`, `expected`, and both smart pointers, so the ownership split shows up as test output and a change of rule shows up as a test failure (see Implementation Experience).

3.5 `reference_or`

`reference_or` computes `common_reference_t<deref_t<T>, U&&>` and, when that type is a reference, binds the result to the contained value or to the alternative without copying either. Const qualification flows through the computation as it should: a `const optional<int>` or a `const int&` alternative each force `const int&` as the result; an `int*` with an `int&` alternative yields `int&`, writable through, and mutation through the result reaches the original object. The function introduces no copies.

Testing surfaced a subtlety, however: the common reference of two unrelated lvalue references need not be a reference. `common_reference_t<int&, long&>` is `long`, and `common_reference_t<string&, const char(&)[2]>` is `string`; in such calls `reference_or` compiles, the dangling guards are vacuously satisfied (`R` is not a reference, so nothing can bind to a temporary) and the function quietly returns a prvalue, behaving like `value_or`. No lifetime hazard arises, and this is the conditional expression's own behavior faithfully reproduced; but a caller who wrote `reference_or` for its reference semantics receives a copy. Whether `reference_or` should further mandate that the computed type *is* a reference type, making the degenerate calls ill-formed and steering them to `value_or`, is the first of the two open questions below.

The danger of returning references is binding one to a temporary, and that is now statically detectable. The implementation carries two guards:

```
static_assert(!std::reference_constructs_from_temporary_v<R, U>);
static_assert(
    !std::reference_constructs_from_temporary_v<R, deref_t<T>>);
```

The first rejects the alternative materializing a temporary: in `reference_or(opt, 0)` the computed common reference of `int&` and `int&&` is `const int&`, which would bind to a temporary materialized from the literal, and the call is ill-formed. The second rejects the symmetric case where the conversion of `*m` to `R` would materialize a temporary. Together they admit exactly the calls whose result refers to an object that outlives the expression, and reject at compile time the calls that core language lifetime rules would quietly accept and then dangle.

Whether these should be `static_asserts` or constraints is a design question for LEWG. As `static_asserts`, a dangling call is a hard, clearly-diagnosed error wherever it occurs, including inside an overload-resolution

context, where it is an error and not a removed candidate. As constraints, the overload disappears SFINAE-style, which composes better with generic code probing for usability but turns a lifetime bug into a silent “no matching function” or, worse, a fallback to a different overload. Our position is that a dangling `reference_or` call is a bug to be reported, not a candidate to be quietly discarded, and we propose the `static_assert` behavior (specified as *Mandates*).

The Tokyo discussion of the P2988 homework concluded that `value_or` is not an available name for a reference-returning function, and suggested `ref_or`. This paper spells it `reference_or`, matching the library’s general preference for unabbreviated names.

3.6 `or_invoke` and laziness

`value_or` and `reference_or` evaluate their alternative unconditionally; it is an ordinary function argument. When the alternative is expensive to construct, that cost is unwelcome on the engaged path, and the established cure is to pass a nullary invocable that is called only when needed. `or_invoke(m, f)` returns `*m` converted to `R` when `m` is engaged and `f()` converted to `R` otherwise, with `R = common_type_t<deref_t<T>, invoke_result_t<I>>`; the invocable is not called on the engaged path.

Because `or_invoke` uses `common_type`, an invocable returning a reference still produces a value: `common_type_t<int&, int&>` is `int`. The family therefore covers three of four quadrants, eager value (`value_or`), eager reference (`reference_or`), and lazy value (`or_invoke`, and `or_construct` below), and omits the fourth. A lazy reference-returning form is specifiable (the dangling guard would apply to `invoke_result_t<I>` just as it does to `U`), but the use case, a reference alternative expensive to *name* and not to construct, is thin, and it is not proposed. If LEWG wants the quadrant filled, it composes naturally as `reference_invoke` later.

The obvious name, `or_else`, is taken: `std::optional::or_else` returns an `optional`, re-wrapping the result, and giving the same name to a value-producing free function would make `x.or_else(f)` and `or_else(x, f)` mean different things. The member half faces the same collision, and answers it as [P3413R0] did, with `value_or_else`. For the free function this paper proposes `or_invoke`. The two names differ because the two functions differ: the member returns `T`, the free function the common type.

3.7 `or_construct`: the fallback as constructor arguments

`or_construct` is the free-function analogue of the other half’s member `value_or_construct`: the alternative is supplied not as a value but as constructor *arguments*, and the fallback object is constructed lazily, only on the disengaged path. On the engaged path nothing is constructed from them at all. An `initializer_list` overload provides the same convenience the member form has.

However, its return-type rule differs from the other three. There is no fallback *value* whose type could participate in a `common_type` computation, only arguments waiting to become one, so by default `R` is the decayed element type itself, `remove_cvref_t<deref_t<T>>`, and the conversion runs *inward*: the arguments construct an `R` directly, and the engaged payload converts to `R`. This is the member function’s model, and here it is the right one; the outward common-type computation is meaningless without a second value.

What the free function can offer that the member cannot is choice of the constructed type. The leading `Ret` template parameter (defaulted through a `void` sentinel) lets the caller name the result type explicitly:

```
std::optional<std::pmr::string> m = lookup(key);
auto s = std::or_construct<std::string>(m, "default");
// std::string either way: converted from the payload, or
// constructed from the arguments --- chosen by the caller
```

The fallback is direct-initialized, so `explicit` constructors are usable; the arguments were always explicitly a construction request. Left to its default, `R` is a decayed object type, so `or_construct` returns a prvalue and can not dangle. With `optional<T&>` the result is a value copy of the referent or a freshly constructed object, never a reference. A caller who names a reference `Ret` explicitly steps outside that guarantee and takes on the lifetime obligation, as `or_construct<const int&>(m, 0)` shows.

3.8 The member half: `value_or_construct` and `value_or_else`

The members are as [P3413R0] specified them, and their design questions are settled by the member they join. Both return `T`, and require the alternative to converge on `T`, because a member named for `value_or`, sitting beside it, would be a trap if it answered with a different type; the corrected result type is the free half’s business, under its own names. Both come in `const&` and `&&` form, so an rvalue moves its payload out. `value_or_construct` takes constructor arguments, with an `initializer_list` overload, and

direct-initializes the fallback, so `explicit` constructors are usable — the arguments were always explicitly a construction request. `value_or_else` takes a nullary invocable whose result must convert to `T`. Neither touches the fallback on the engaged path. `expected<void,E>` gains neither; there is no value to construct.

Two things follow from the halves being one paper. The names do not collide: `x.value_or_construct(args...)` and `or_construct<R>(m, args...)` are different spellings of the same model, and `x.value_or_else(f)` and `or_invoke(x, f)` differ in return type deliberately (see `or_invoke` and laziness, above). And `optional<T&>` no longer has to be deferred: P3413 deferred it to [P2988R12] while that paper was in flight, and `optional<T&>` is in C++26 now, so the members are specified over it here, returning a value, exactly as its `value_or` does. `expected<T&,E>` is the one form still waiting on a paper of its own, and the wording says so.

3.9 Why free functions

The member route is open for one defect and closed for the other, which is why the proposal has two halves rather than one. Adding `value_or_construct` and `value_or_else` adds new names, and new names can be added; that is the member half. Correcting the type of the answer cannot be done that way. For `std::optional` and `std::expected` the existing `value_or` cannot change its return type, and a second member with corrected semantics would have to coexist with the first forever, on each class, kept in sync by hand. For a raw pointer there is nowhere to put a member at all, and adding one to each smart pointer would repeat the same hand-synchronization. A constrained free function template is the only form that can state the semantics once and apply them to every type speaking the protocol, including user-defined nullables, which model the concept simply by being contextually convertible to `bool` and dereferenceable.

These are ordinary function templates rather than customization point objects. There is nothing to customize: the semantics are fully determined by the protocol, and a type that wants different behavior for “value or default” is not modeling nullable, it is doing something else. The usual argument for a CPO, dispatching to a type’s own implementation, is therefore absent, and the names do not inhibit ADL.

3.10 `std::expected`, and the error

`std::expected<T,E>` models nullable: both `bool(e)` and `*e` are valid on a const `expected`. Consequently `value_or(e, u)` works, and discards the error, as the member `value_or` does. This is by design: the moment the question is “the value, or this instead,” the error has been decided to be uninteresting. Uses that need the error have `error_or` and the monadic interface on the member side; no error-observing overload is proposed here.

3.11 What about `yield_if`, and the end of P1255

[P2988R12] forecast a fourth function, `yield_if`, producing a view of zero or one elements, alongside the three proposed here. It was to remain with the [P1255R13] ranges work. However, that work is concluded. Range support for `std::optional` [P3168R2] supplanted `views::maybe`, and `views::nullable` with it: an optional now *is* the view of zero or one elements, and a pointer-like nullable reaches the same place through `optional<T&>` [P2988R12], as `p ? optional<T&>(*p) : nullopt`. Downey, its author, does not expect to revise P1255 further; this paper inherits its remaining deliverables, the concept and the free functions over it.

That covers `yield_if` as well: `yield_if(cond, value)` is now spellable as `cond ? optional(value) : nullopt`, which is itself a range. What residual demand exists is for a named convenience in comprehension-style pipelines; if implementation experience shows it is wanted, it is a small separate paper. It is explicitly not part of this proposal.

4 P3413R0, and the two halves

[P3413R0] (“A more flexible `optional::value_or` (else!)”), by Corentin Jabot, reviving [P2218R0], was a separate paper. It proposed the member `value_or_construct`, constructing the fallback `T` lazily and in place from forwarded arguments, with `initializer_list` overloads, and `value_or_else`, taking a lazily evaluated nullary callable, on both `std::optional` and `std::expected`. Reviewing R0 of this paper in Brno, LEWG saw the two as fixes to the same member function and asked, “Can we ask you and Corentin to come back with one proposal?” This is that proposal. Corentin is a co-author here, and P3413’s members, wording, and rationale are the member half of this document; P3413R0 is not pursued separately.

Unification changed presentation, not design. The members are as P3413 specified them, with the one substantive addition the intervening time allows: P3413 deferred `optional<T&>` to [P2988R12] while that paper was in flight, and `optional<T&>` is in C++26 now, so the members are specified over it, and over the

reference specializations of `expected` as far as the working draft allows. The free half is untouched by the unification; where it has changed since R0 — the `rvalue` rule of Design — it changed for its own reasons.

What each half fixes stays distinct, and the distinction is worth holding on to in review. The member half fixes the *cost* of the alternative: with `value_or_construct` the fallback object is not constructed at all on the engaged path, and `value_or_else` defers arbitrary computation. It does not touch the return type; everything still returns `T`, with the alternative still required to convert to `T`, so the truncation pitfall, the forced copies, and the impossibility of reference results all remain — deliberately, since a member beside `value_or` has to behave like `value_or`. It is also necessarily member-by-member: `optional` and `expected` each grow six functions, and pointers of any kind get nothing.

The free half fixes the *type* of the answer and its *reach*: results are computed via `common_type/common_reference` instead of funneled through `T`, reference results become possible and dangling-checked, and one set of free functions covers every nullable type. The lazy cases are where the halves meet. `or_invoke` and member `value_or_else` differ in form (free vs. member), reach (any nullable vs. `optional/expected`), and return type (`common_type` vs. `T`). `or_construct` and member `value_or_construct` share the in-place, lazy construction model and the `initializer_list` overload; the free function also lets the caller name the constructed type (`or_construct<string>(m, args...)`) and works on every nullable.

	member half (P3413R0)	free half
form	members	free functions
applies to	<code>optional</code> , <code>expected</code>	any nullable (incl. pointers)
return type	<code>T</code>	<code>common_type</code> / <code>common_reference</code>
truncation pitfall	unchanged, by design	fixed
reference results	no	<code>reference_or</code> , dangling-checked
lazy alternative	<code>value_or_else</code>	<code>or_invoke</code>
in-place fallback construction	<code>value_or_construct</code>	<code>or_construct</code> , caller-chosen result type
rvalue nullable moves payload	yes (&& overloads)	yes, for the owning models (see Design)
<code>optional<T&></code>	members return a value	covered by the concept

The halves compose. Adopt the members and `optional` and `expected` gain the lazy, in-place fallback, spelled the way a member is spelled and moving out of an rvalue. Adopt the free functions and the corrected semantics — the computed result type, reference results, the reach to pointers and every other nullable — become available generically, over those same types and over the ones that have no members to grow. A caller writing `x.value_or_construct(args...)` and one writing `or_construct<R>(m, args...)` are served by the same design at two spellings. The halves also remain severable: LEWG can take one and leave the other, and each stands alone. What unification bought is that they need no longer be weighed against each other.

5 Principles for Reification of Design

The same principles that governed [P2988R12] govern here.

Errors that can be detected at compile time must be detected at compile time. The concept rejects non-nullable arguments at the constraint; `reference_or` rejects every call whose result would bind to a temporary via `reference_constructs_from_temporary_v`, on both the alternative path and the contained-value path.

Never knowingly produce a dangling reference. The functions take no address and extend no lifetime; `reference_or`'s result always refers to an object that existed before the call. The second guard is stated on `deref_t<T>`, the type `*m` yields; stated on `T&` it would name the nullable rather than its payload, and could never fire. The rejected cases are those where the core language would have materialized a temporary: a prvalue alternative whose common reference with the dereference type is a const reference (`reference_or(opt, 0)`), and conversions of the contained value to the result type that materialize.

Value-producing functions never dangle by construction. `value_or` and `or_invoke` return prvalues of a decayed common type; there is no reference in their result to check.

These principles are enforced by a negative-compilation test suite in the reference implementation: each forbidden combination is a translation unit that must fail to compile, with the diagnostic matched against the constraint or `static_assert` that is required to fire.

6 Implementation Experience

The reference implementation is `beman.free_value_or` [Downey_free_value_or]. The implementation floor is C++23, required for `reference_constructs_from_temporary_v`; the functions are otherwise C++20 material.

The test suite exercises the matrix of: nullable types `std::optional<T>`, `std::expected<T,E>`, `T*`, `std::shared_ptr<T>`, `std::unique_ptr<T>`, and (gated on a C++26 toolchain, via the Beman `optional` reference implementation of P2988 [The_Beman_Project_beman_optional]) `std::optional<T&>`; engaged and disengaged; the nullable as lvalue, const lvalue, and rvalue, the rvalue cases copy- and move-counted on the engaged path so that the ownership split is pinned rather than assumed; the alternative as lvalue and rvalue; mixed-type combinations; constant evaluation of all four functions; laziness of `or_invoke` and `or_construct` and eagerness of `value_or`; for `or_construct`, construction arities from zero arguments through multi-argument emplacement and the `initializer_list` overload, with construction confirmed to occur only on the disengaged path; and negative-compilation cases for both concept violations and dangling rejection. Reference identity and mutation round-trips confirm that `reference_or` introduces no copies: writes through its result reach the contained object when engaged and the alternative when not. The `optional<T&>` suite additionally verifies rebinding semantics: after an `optional<T&>` is rebound to a new referent, `value_or` produces and `reference_or` refers to the new referent, and disengaging falls through to the alternative.

The suite, at this writing 118 test cases and nine negative-compilation cases over eight translation units, all green, found no defects in the implementation. It did surface two behaviors reported in Design: that `reference_or` degrades to a prvalue result when the common reference of unrelated types is not a reference type, with the dangling guards then vacuous; and that `or_invoke`'s use of `common_type` decays reference-returning invocables to values, leaving the lazy-reference quadrant of the family empty by choice. The negative-compilation harness was itself validated: each WILL-FAIL translation unit was checked to fail for the required reason by mutating its expected-diagnostic pattern and observing the test fail. The rvalue cases were validated the same way, against the library and not the harness. Reverting the dereference type to `std::iter_reference_t<T>`, the observe-only alternative, makes `reference_or`'s static assertions fail to compile and, with those removed so that they cannot mask the rest, fails the move counts for `value_or` over `optional` and over `expected`, for `or_invoke`, and for `or_construct`. It leaves the smart pointer cases passing, because those copy under both rules. That is the ownership split of Design showing up as test output. Coverage of the header from that one file is complete on lines, functions, and branches.

Verified `common_reference` results for `reference_or`, showing const propagation:

nullable (lvalue)	alternative (lvalue)	result
<code>optional<int>&</code>	<code>int&</code>	<code>int&</code>
<code>const optional<int>&</code>	<code>int&</code>	<code>const int&</code>
<code>optional<int>&</code>	<code>const int&</code>	<code>const int&</code>
<code>expected<int,int>&</code>	<code>int&</code>	<code>int&</code>
<code>int*</code>	<code>int&</code>	<code>int&</code>
<code>shared_ptr<int>&</code>	<code>int&</code>	<code>int&</code>
<code>optional<string>&</code>	<code>string&</code>	<code>string&</code>

The current implementation, less the portability shims — a polyfill over the compiler builtin for `reference_constructs_from_temporary_v`, and a template parameter order that MSVC accepts:

```
template <class T>
concept nullable = requires(const std::remove_reference_t<T>& t) {
    bool(t);
    *(t);
};

// The type *m yields, carrying the value category of the nullable through
// to its payload. Exposition only in the wording.
template <class T>
using deref_t = decltype(*std::declval<T>());

template <nullable T, class U,
        class R = std::common_reference_t<deref_t<T>, U&&>>
constexpr auto reference_or(T&& m, U&& u) -> R {
```

```

    static_assert(!std::reference_constructs_from_temporary_v<R, U>);
    static_assert(
        !std::reference_constructs_from_temporary_v<R, deref_t<T>>);
    return bool(m) ? static_cast<R>(*m) : static_cast<R>((U&&)u);
}

template <nullable T, class U,
         class R = std::common_type_t<deref_t<T>, U&&>>
constexpr auto value_or(T&& m, U&& u) -> R {
    return bool(m) ? static_cast<R>(*m) : static_cast<R>(std::forward<U>(u));
}

template <nullable T, class I,
         class R = std::common_type_t<deref_t<T>,
                                     std::invoke_result_t<I>>>
constexpr auto or_invoke(T&& m, I&& invocable) -> R {
    return bool(m) ? static_cast<R>(*m)
        : static_cast<R>(std::forward<I>(invocable)());
}

template <class Ret = void, nullable T, class... Args,
         class R = std::conditional_t<std::is_void_v<Ret>,
                                     std::remove_cvref_t<deref_t<T>>, Ret>>
constexpr R or_construct(T&& m, Args&&... args) {
    return bool(m) ? static_cast<R>(*m) : R(std::forward<Args>(args)...);
}

template <class Ret = void, nullable T, class E, class... Args,
         class R = std::conditional_t<std::is_void_v<Ret>,
                                     std::remove_cvref_t<deref_t<T>>, Ret>>
constexpr R or_construct(T&& m, std::initializer_list<E> il,
                        Args&&... args) {
    return bool(m) ? static_cast<R>(*m) : R(il, std::forward<Args>(args)...);
}

```

The member side is implemented too. `value_or_construct` and `value_or_else` were added to the Beman reference implementations of `optional` [The_Beman_Project_beman_optional] and `expected` [The_Beman_Project_beman_expected], vendored into `beman.free_value_or` and edited in place, on both the primary templates and their reference specializations. `beman.expected` already carries `expected<T&,E>`, `expected<T,E&>`, and `expected<T&,E&>`, so the members run over references now, ahead of any such paper reaching the working draft; standardizing those forms is the only dependency, and not implementing them. The two libraries' own suites, GoogleTest for `optional` and Catch2 for `expected`, run as part of this project's tests, and cover laziness (a construction counter that never ticks on the engaged path), the `initializer_list` overload, rvalue move-out, constant evaluation, and the reference specializations. All green.

7 Proposal

Add to the standard library four constrained free function templates, `value_or`, `reference_or`, `or_invoke`, and `or_construct`, and the named concept `nullable` that constrains them, with the semantics described above, targeting C++29. The functions should be usable on any type that is contextually convertible to `bool` and dereferenceable, including `std::optional`, `std::expected`, raw pointers, `std::shared_ptr`, `std::unique_ptr`, and `std::optional<T&>`.

Add the member functions of the other half, as [P3413R0] specified them: `value_or_construct`, with its `initializer_list` overload, and `value_or_else`, on both `std::optional` and `std::expected`, each in `const&` and `&&` form, producing the fallback lazily and in place. Their wording is reproduced below, unchanged in substance from P3413. `std::optional<T&>` [P2988R12] is in C++26, and the members apply to it directly, returning a value; a future `std::expected<T&,E>` would carry them the same way, and that one case depends on the expected-over-references work, as the wording notes. The two feature sets are severable: LEWG may take the free functions, the members, or both, but the point of unifying is that it need not.

One question is genuinely open, and we ask LEWG to answer it before wording review: whether `reference_or` should mandate that the computed common reference *is* a reference type. As specified it silently degrades to a prvalue when the payload and the alternative are unrelated — `reference_or` over an `optional<int>` and a `long&` yields a `long` — so a caller who reached for it to avoid a copy gets one, and writes through the result do not reach the original object (see Design).

The rest we have decided, and will revisit only if LEWG disagrees. An rvalue nullable moves its payload, which divides the models along ownership: `optional` and `expected` own their value and move it, while `T*`, the smart pointers, and `optional<T&>` are handles to a referent that outlives them and go on copying. The alternative is to dereference an lvalue always, so that the functions observe and never consume; it is simpler to state, and it makes every owning nullable copy a payload that is going away. We would rather be correct by default, and the alternative is a one-line change to the dereference type if LEWG prefers it (see Design). The dangling guards are *Mandates*, so a dangling call is diagnosed and not removed from the overload set. `nullable` is a public concept. The names are `value_or`, `reference_or`, `or_invoke`, and `or_construct`. The concept stays minimal, so a function pointer models it and is rejected downstream when `common_type` fails, and not at the constraint. The functions do not inhibit ADL. They should be freestanding, and we propose `<optional>` as the header, where users will look for them.

8 Wording

Initial draft wording, relative to [N5054], to be completed after LEWG design review. Shown in `<optional>`.

Add to the header synopsis:

```
namespace std {
    template<class T>
        concept nullable = requires(const remove_reference_t<T>& t) {
            bool(t);
            *(t);
        };

    template<nullable T, class U>
        constexpr common_reference_t<deref-t<T>, U&&>
            reference_or(T&& m, U&& u);

    template<nullable T, class U>
        constexpr common_type_t<deref-t<T>, U&&>
            value_or(T&& m, U&& u);

    template<nullable T, class I>
        constexpr common_type_t<deref-t<T>, invoke_result_t<I>>
            or_invoke(T&& m, I&& invocable);

    template<class Ret = void, nullable T, class... Args>
        constexpr conditional_t<is_void_v<Ret>,
                                remove_cvref_t<deref-t<T>>, Ret>
            or_construct(T&& m, Args&&... args);

    template<class Ret = void, nullable T, class E, class... Args>
        constexpr conditional_t<is_void_v<Ret>,
                                remove_cvref_t<deref-t<T>>, Ret>
            or_construct(T&& m, initializer_list<E> il, Args&&... args);
}
```

¹ Let `t` be a const lvalue of type `remove_reference_t<T>`. `T` models `nullable` only if:

- (1.1) — `bool(t)` is equality-preserving and does not modify `t`;
- (1.2) — `*t` is valid only when `bool(t)` is `true`; and
- (1.3) — `*t` is equality-preserving and does not modify `t`.

```
template<nullable T, class U>
    constexpr common_reference_t<deref-t<T>, U&&>
        reference_or(T&& m, U&& u);
```

2 Let R be `common_reference_t<@deref-t@<T>, U&&>`.

3 *Mandates:* `reference_constructs_from_temporary_v<R, U>` is false and `reference_constructs_from_temporary_v<R, @deref-t@<T>>` is false.

4 *Effects:* Equivalent to: `return bool(m) ? static_cast<R>(*m) : static_cast<R>(std::forward<U>(u));`
`template<nullable T, class U>`
`constexpr common_type_t<deref-t<T>, U&&>`
`value_or(T&& m, U&& u);`

5 Let R be `common_type_t<@deref-t@<T>, U&&>`.

6 *Effects:* Equivalent to: `return bool(m) ? static_cast<R>(*m) : static_cast<R>(std::forward<U>(u));`
`template<nullable T, class I>`
`constexpr common_type_t<deref-t<T>, invoke_result_t<I>>`
`or_invoke(T&& m, I&& invocable);`

7 Let R be `common_type_t<@deref-t@<T>, invoke_result_t<I>>`.

8 *Effects:* Equivalent to: `return bool(m) ? static_cast<R>(*m) : static_cast<R>(std::forward<I>(invocable));`
`template<class Ret = void, nullable T, class... Args>`
`constexpr conditional_t<is_void_v<Ret>,`
`remove_cvref_t<deref-t<T>>, Ret>`
`or_construct(T&& m, Args&&... args);`

9 Let R be `conditional_t<is_void_v<Ret>, remove_cvref_t<@deref-t@<T>>, Ret>`.

10 *Mandates:* `is_constructible_v<R, Args...>` is true and `is_convertible_v<@deref-t@<T>, R>` is true.

11 *Effects:* Equivalent to: `return bool(m) ? static_cast<R>(*m) : R(std::forward<Args>(args)...);`
`template<class Ret = void, nullable T, class E, class... Args>`
`constexpr conditional_t<is_void_v<Ret>,`
`remove_cvref_t<deref-t<T>>, Ret>`
`or_construct(T&& m, initializer_list<E> il, Args&&... args);`

12 Let R be `conditional_t<is_void_v<Ret>, remove_cvref_t<@deref-t@<T>>, Ret>`.

13 *Mandates:* `is_constructible_v<R, initializer_list<E>&, Args...>` is true and `is_convertible_v<@deref-t@<T>, R>` is true.

14 *Effects:* Equivalent to: `return bool(m) ? static_cast<R>(*m) : R(il, std::forward<Args>(args)...);`

8.1 Member functions, from P3413R0

The following member wording is reproduced from [P3413R0], relative to [N5054], and is proposed jointly with the free functions above. It is purely additive: the existing `value_or` is unchanged. Return types and *Effects* follow P3413; the *Mandates* spell the construction requirement as `is_constructible_v<T, Args...>`, matching the *Effects*.

8.1.1 optional [optional.optional], [optional.observe]

To the synopsis of `optional` in [optional.optional.general], after the `value_or` declarations, add:

```
template<class... Args> constexpr T value_or_construct(Args&&... args) const&
template<class... Args> constexpr T value_or_construct(Args&&... args) &&

template<class U, class... Args>
constexpr T value_or_construct(initializer_list<U> il, Args&&... args) const&
template<class U, class... Args>
constexpr T value_or_construct(initializer_list<U> il, Args&&... args) &&

template<class F> constexpr T value_or_else(F&& f) const&
template<class F> constexpr T value_or_else(F&& f) &&
```

To [optional.observe], add:

```
template<class... Args> constexpr T value_or_construct(Args&&... args) const&
```

- 1 *Mandates:* `is_copy_constructible_v<T>` is true and `is_constructible_v<T, Args...>` is true.
Effects: Equivalent to: `return has_value() ? **this : T(std::forward<Args>(args)...);`
`template<class... Args> constexpr T value_or_construct(Args&&... args) &&;`
- 2 *Mandates:* `is_move_constructible_v<T>` is true and `is_constructible_v<T, Args...>` is true.
Effects: Equivalent to: `return has_value() ? std::move(**this) : T(std::forward<Args>(args)...);`
`template<class U, class... Args>`
`constexpr T value_or_construct(initializer_list<U> il, Args&&... args) const&;`
- 3 *Mandates:* `is_copy_constructible_v<T>` is true and `is_constructible_v<T, initializer_list<U>&, Args...>` is true. *Effects:* Equivalent to: `return has_value() ? **this : T(il, std::forward<Args>(args)...);`
`template<class U, class... Args>`
`constexpr T value_or_construct(initializer_list<U> il, Args&&... args) &&;`
- 4 *Mandates:* `is_move_constructible_v<T>` is true and `is_constructible_v<T, initializer_list<U>&, Args...>` is true. *Effects:* Equivalent to: `return has_value() ? std::move(**this) : T(il, std::forward<Args>(args)...);`
`template<class F> constexpr T value_or_else(F&& f) const&;`
- 5 Let U be `invoke_result_t<F>`. *Mandates:* `is_copy_constructible_v<T>` is true and `is_convertible_v<U, T>` is true. *Effects:* Equivalent to: `return has_value() ? **this : std::forward<F>(f)();`
`template<class F> constexpr T value_or_else(F&& f) &&;`
- 6 Let U be `invoke_result_t<F>`. *Mandates:* `is_move_constructible_v<T>` is true and `is_convertible_v<U, T>` is true. *Effects:* Equivalent to: `return has_value() ? std::move(**this) : std::forward<F>(f)();`
- 7 The reference specialization `optional<T&>` [P2988R12] carries these members too; its wording is given together with `expected<T&,E>` below.

8.1.2 expected [expected.expected], [expected.object.obs]

To the synopsis of `expected` in [expected.expected.general], after the `value_or` declarations, add:

```
template<class... Args> constexpr T value_or_construct(Args&&... args) const&;
template<class... Args> constexpr T value_or_construct(Args&&... args) &&;

template<class U, class... Args>
constexpr T value_or_construct(initializer_list<U> il, Args&&... args) const&;
template<class U, class... Args>
constexpr T value_or_construct(initializer_list<U> il, Args&&... args) &&;

template<class F> constexpr T value_or_else(F&& f) const&;
template<class F> constexpr T value_or_else(F&& f) &&;
```

To [expected.object.obs], add:

- `template<class... Args> constexpr T value_or_construct(Args&&... args) const&;`
- 1 *Mandates:* `is_copy_constructible_v<T>` is true and `is_constructible_v<T, Args...>` is true.
Effects: Equivalent to: `return has_value() ? **this : T(std::forward<Args>(args)...);`
`template<class... Args> constexpr T value_or_construct(Args&&... args) &&;`
- 2 *Mandates:* `is_move_constructible_v<T>` is true and `is_constructible_v<T, Args...>` is true.
Effects: Equivalent to: `return has_value() ? std::move(**this) : T(std::forward<Args>(args)...);`
`template<class U, class... Args>`
`constexpr T value_or_construct(initializer_list<U> il, Args&&... args) const&;`
- 3 *Mandates:* `is_copy_constructible_v<T>` is true and `is_constructible_v<T, initializer_list<U>&, Args...>` is true. *Effects:* Equivalent to: `return has_value() ? **this : T(il, std::forward<Args>(args)...);`
`template<class U, class... Args>`
`constexpr T value_or_construct(initializer_list<U> il, Args&&... args) &&;`
- 4 *Mandates:* `is_move_constructible_v<T>` is true and `is_constructible_v<T, initializer_list<U>&, Args...>` is true. *Effects:* Equivalent to: `return has_value() ? std::move(**this) : T(il, std::forward<Args>(args)...);`

```

    template<class F> constexpr T value_or_else(F&& f) const&&;
5   Let U be invoke_result_t<F>. Mandates: is_copy_constructible_v<T> is true and is_convertible_v<U, T> is true. Effects: Equivalent to: return has_value() ? *this : T(std::forward<F>(f)());
    template<class F> constexpr T value_or_else(F&& f) &&;
6   Let U be invoke_result_t<F>. Mandates: is_move_constructible_v<T> is true and is_convertible_v<U, T> is true. Effects: Equivalent to: return has_value() ? std::move(*this) : T(std::forward<F>(f)());
7   expected<void,E> gains neither member — there is no value to construct — and no error_or_construct or error_or_else is proposed, following [P3413R0].

```

8.1.3 Reference specializations, and the expected-over-references dependency

`std::optional<T&>` [P2988R12] is in C++26; `std::expected<T&,E>` is not standardized yet, but is expected to follow. On both, these members return a value rather than a reference — exactly as `value_or` does on those specializations — and are declared `const`, there being no rvalue overload to distinguish. For `optional<T&>` and `expected<T&,E>`, add:

```

    template<class... Args>
        constexpr remove_cv_t<T> value_or_construct(Args&&... args) const;
    template<class U, class... Args>
        constexpr remove_cv_t<T> value_or_construct(initializer_list<U> il, Args&&... args) const;
    template<class F>
        constexpr remove_cv_t<T> value_or_else(F&& f) const;
1   Let X be remove_cv_t<T>, and for value_or_else let U be invoke_result_t<F>. Mandates: is_convertible_v<T&, X> is true; for value_or_construct, is_constructible_v<X, Args...> (respectively is_constructible_v<X, initializer_list<U>&, Args...>) is true; for value_or_else, is_convertible_v<U, X> is true. Effects: Equivalent to, respectively:
2   return has_value() ? *this : X(std::forward<Args>(args)...);
    return has_value() ? *this : X(il, std::forward<Args>(args)...);
    return has_value() ? *this : std::forward<F>(f)();
3   The optional<T&> wording above is a plain addition to a C++26 facility; nothing about it is contingent. Only the expected<T&,E> case is: it depends on the expected-over-references work, which is not yet in the working draft. Both reference forms are implemented already — the Beman expected carries expected<T&,E>, expected<T,E&>, and expected<T&,E&>, and the Beman optional the adopted optional<T&> — so they are exercised now, not merely specified (see Implementation Experience).

```

8.2 Feature test macro

Add a new feature test macro `__cpp_lib_value_or`, set to the date of adoption, for the free functions. For the member functions, bump `__cpp_lib_optional` and `__cpp_lib_expected` to the date of adoption, per [P3413R0]; these are independent of each other and of the `optional<T&>` macro.

9 Impact on the standard

A pure library extension. The free functions change no existing class or function. The member functions add `value_or_construct` and `value_or_else` to `optional` and `expected` without altering any existing member, so nothing existing changes meaning; being member function templates, they add nothing to the ABI. The free functions require C++23 library support (`reference_constructs_from_temporary_v`); the members impose no requirement beyond the class they are added to.

10 Acknowledgments

Thanks to Peter Sommerlad, co-author of [P2988R12], where the member `value_or` question was fought to a standstill, and to the LEWG reviewers of P1255 and P2988 whose objections shaped these functions. The Tokyo review homework on `value_or` alternatives surveyed the implementation divergence reported here. Thanks to Marc Mutz, whose [P2218R0] originated the member `value_or_construct` and `value_or_else` that [P3413R0] revived, and to Barry Revzin for the wording feedback carried through that paper.

11 Document history

- **R1** 2026-09-10 — Unified with [P3413R0] following LEWG’s Brno review, which asked for a single proposal. Corentin Jabot joins as co-author; the members `value_or_construct` and `value_or_else` are folded into the proposal, wording, and implementation, and specified over `optional<T&>`, which P3413R0 deferred. Separately: an rvalue nullable moves its payload, splitting the models by ownership, and the asks to LEWG are cut to two.
- **R0** Brno, 2026-06 — Initial draft, reviewed by LEWG. The free-function follow-on promised in [P2988R12], concluding the line of work begun in [P1255R13].

References

- [Downey_free_value_or] Stephen Downey. `beman.free_value_or`. https://github.com/steve-downey/free_value_or.
- [N5054] Thomas Köppe. N5054: Working draft, programming languages — c++. <https://wg21.link/n5054>, 7 2026.
- [P1255R13] Steve Downey. P1255R13: A view of 0 or 1 elements: `views::nullable` and a concept to constrain maybes. <https://wg21.link/p1255r13>, 5 2024.
- [P2218R0] Marc Mutz. P2218R0: More flexible `optional::value_or()`. <https://wg21.link/p2218r0>, 9 2020.
- [P2255R2] Tim Song. P2255R2: A type trait to detect reference binding to temporary. <https://wg21.link/p2255r2>, 10 2021.
- [P2988R12] Steve Downey and Peter Sommerlad. P2988R12: `std::optional<t&>`. <https://wg21.link/p2988r12>, 4 2025.
- [P3168R2] David Sankel, Marco Foco, Darius Neațu, and Barry Revzin. P3168R2: Give `std::optional` range support. <https://wg21.link/p3168r2>, 6 2024.
- [P3413R0] Corentin Jabot. P3413R0: A more flexible `optional::value_or` (else!). <https://wg21.link/p3413r0>, 10 2024.
- [The_Beman_Project_beman_expected] The Beman Project. `beman.expected`. <https://github.com/bemanproject/expected>.
- [The_Beman_Project_beman_optional] The Beman Project. `beman.optional`. <https://github.com/bemanproject/optional>.